

PROJECT

PREDICTING THE LIKELIHOOD OF HEART DISEASE FROM PATIENTS

PROJECT UNDERSTANDING

Project Overview

The primary objective of this capstone project is to create a predictive model that assesses the likelihood of heart disease in patients based on a variety of health-related features. The model will leverage data from the "heart_disease.csv" dataset, which includes demographic information, clinical measurements, and historical health data.

Problem Statement

Cardiovascular disease (CVD) is the leading global cause of death, disproportionately affecting low- and middle-income countries, where it accounts for three-quarters of all fatalities (Kontchou, McCracy, & Schulman, 2019). In Kenya, CVD remains a significant public health burden, with a mortality rate of 13.8% in 2019. The disease primarily affects individuals aged 30–70 years, with at least 2.5 heart surgeries performed per 1,000 individuals at a corresponding labor cost of US\$526.

Despite growing awareness, early detection and risk assessment remain challenging due to limited local research and data-driven insights. Rapid urbanization, lifestyle changes, and inadequate access to preventive healthcare further exacerbate the situation. This study aims to analyze key health indicators, identify major risk factors, and uncover trends to support healthcare professionals, policymakers, and researchers. By leveraging data-driven insights, this research seeks to improve early diagnosis, develop targeted interventions, and enhance public health strategies to combat heart disease in Kenya.

General Objectives

1. Developing a Predictive Model:

- Objective: Create a machine learning model that accurately predicts the likelihood of heart disease based on patient data.
- Key Performance Indicators (KPIs): Model accuracy, precision, recall, F1-score, and area under the ROC curve (AUC-ROC).
- Outcome: A validated model that can be used in clinical settings to assess patient risk.

2. Identify Key Risk Factors:

- Objective: Analyze the dataset to determine which features are the most significant predictors of heart disease.
- KPIs: Identification of top risk factors, correlation coefficients, and feature importance scores.
- Outcome: A report detailing the primary risk factors associated with heart disease, which can inform patient education and preventive strategies.

3. Enhance Patient Care:

- Objective: Provide healthcare professionals with a tool that aids in the early detection and management of heart disease.
- KPIs: User satisfaction ratings, number of healthcare providers using the tool, and feedback from clinical trials.
- Outcome: Improved patient outcomes through timely interventions based on risk assessments generated by the model.

4. Facilitate Data-Driven Decision Making:

- Objective: Enable healthcare organizations to make informed decisions regarding resource allocation and preventive care strategies based on model predictions.
- KPIs: Changes in resource allocation, reduction in hospital admissions for heart disease, and improved health outcomes in high-risk populations.
- Outcome: A framework for integrating predictive analytics into healthcare decision-making processes.

5. Raise Awareness and Educate Patients:

- Objective: Increase awareness of heart disease risk factors among patients and the general public through educational materials and outreach programs.
- KPIs: Number of educational sessions conducted, patient engagement metrics, and changes in patient knowledge and behavior regarding heart health.
- Outcome: A more informed patient population that actively participates in their health management, leading to a reduction in heart disease prevalence.

Expected Outcomes

The expected outcomes of this project include:

- A Predictive Model: A machine learning model that accurately predicts the likelihood of heart disease based on input features, with a focus on achieving high accuracy and reliability.
- Insights into Risk Factors: A comprehensive analysis of the most significant risk factors associated with heart disease, which can inform both clinical practice and public health initiatives.
- User -Friendly Tool: A prototype or application that allows healthcare providers to input patient data and receive risk assessments, facilitating easier integration into clinical workflows.
- Documentation and Reporting: A detailed report summarizing the methodology, findings, and recommendations for future research or implementation in healthcare settings.

Metrics of success (benefits)

- Early Detection and Prevention- will help identify individuals who are at high risk of heart diseases before symptoms appear. This will also lead to encouraging early lifestyle changes to prevent disease progression and access the right medication which reduces the long run costs.
- Personalized Healthcare-Helps doctors and patients make informed decisions based on personal risk factors and make proper judgement on tests and screenings to be done.
- Cost Savings in Healthcare- By detecting this symptoms early and managing risk factors, you reduce hospital admissions.Lowers the burden on insurance and healthcare systems by minimizing costly late-stage treatments.
- Research and Development- A predictive model can aid in medical research by identifying new patterns in heart disease risks.Provides insights into key factors contributing to heart disease, leading to better prevention strategies.
- Data-Driven Public Health Strategies- Helps governments and healthcare organizations design policies to reduce heart disease prevalence and allow government to make better budget allocations to deal with these cases.

DATA UNDERSTANDING

Importing Libraries

```
In [2]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, classification_report, roc_auc_score
from sklearn.pipeline import Pipeline
from sklearn.model_selection import cross_val_score
import numpy as np
from sklearn.svm import SVC
from imblearn.over_sampling import SMOTE
# Ignore warnings
warnings.filterwarnings("ignore")
```

Loading our data

```
In [3]: data = pd.read_csv("heart_disease.csv")
```

In [4]:

checking the first few rows
data.head()

Out[4]:

	Age	Gender	Blood Pressure	Cholesterol Level	Exercise Habits	Smoking	Family Heart Disease	Diabetes	BMI
0	56.0	Male	153.0	155.0	High	Yes	Yes	No	24.991591
1	69.0	Female	146.0	286.0	High	No	Yes	Yes	25.221799
2	46.0	Male	126.0	216.0	Low	No	No	No	29.855447
3	32.0	Female	122.0	293.0	High	Yes	Yes	No	24.130477
4	60.0	Male	166.0	242.0	Low	Yes	Yes	Yes	20.486289

5 rows × 21 columns

In [5]:

checking the last few rows
data.tail()

Out[5]:

	Age	Gender	Blood Pressure	Cholesterol Level	Exercise Habits	Smoking	Family Heart Disease	Diabetes	BM
9995	25.0	Female	136.0	243.0	Medium	Yes	No	No	18.78879
9996	38.0	Male	172.0	154.0	Medium	No	No	No	31.85680
9997	73.0	Male	152.0	201.0	High	Yes	No	Yes	26.89991
9998	23.0	Male	142.0	299.0	Low	Yes	No	Yes	34.96402
9999	38.0	Female	128.0	193.0	Medium	Yes	Yes	Yes	25.11129

5 rows × 21 columns

```
In [6]: # checking the number of columns and rows in our data
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 21 columns):
 #   Column                                Non-Null Count  Dtype
---  -
 0   Age                                   9971 non-null   float64
 1   Gender                               9981 non-null   object
 2   Blood Pressure                       9981 non-null   float64
 3   Cholesterol Level                    9970 non-null   float64
 4   Exercise Habits                      9975 non-null   object
 5   Smoking                             9975 non-null   object
 6   Family Heart Disease                 9979 non-null   object
 7   Diabetes                             9970 non-null   object
 8   BMI                                  9978 non-null   float64
 9   High Blood Pressure                  9974 non-null   object
10   Low HDL Cholesterol                 9975 non-null   object
11   High LDL Cholesterol                9974 non-null   object
12   Alcohol Consumption                 9968 non-null   object
13   Stress Level                        9978 non-null   object
14   Sleep Hours                         9975 non-null   float64
15   Sugar Consumption                   9970 non-null   object
16   Triglyceride Level                  9974 non-null   float64
17   Fasting Blood Sugar                 9978 non-null   float64
18   CRP Level                           9974 non-null   float64
19   Homocysteine Level                 9980 non-null   float64
20   Heart Disease Status                10000 non-null  object
dtypes: float64(9), object(12)
memory usage: 1.6+ MB
```

Our data has a 21 columns and 10,000 rows.

Columns Descriptions

- Age; Represents the age of the individual in years. Age is a significant risk factor for heart disease, with older individuals generally at higher risk.
- Gender; Indicates the gender of the individual, typically coded as 0 for female and 1 for male. Gender can influence the risk of heart disease due to biological and lifestyle factors.
- Blood Pressure; Refers to the individual's blood pressure reading, usually measured in mmHg (millimeters of mercury). High blood pressure (hypertension) is a major risk factor for heart disease.
- Cholesterol Level; Represents the total cholesterol level in the blood, measured in mg/dL (milligrams per deciliter). High cholesterol levels can lead to plaque buildup in arteries, increasing heart disease risk.
- Exercise Habit; Indicates the frequency or quality of physical activity the individual engages in. Regular exercise is associated with a lower risk of heart disease.
- Smoking; Indicates whether the individual smokes tobacco, often coded as 0 for non-smokers and 1 for smokers. Smoking is a well-known risk factor for heart disease.
- Family Heart Disease; Indicates whether there is a family history of heart disease, typically coded as 0 for no family history and 1 for a positive family history. A family history can

increase an individual's risk.

- Diabetes; Indicates whether the individual has diabetes, often coded as 0 for non-diabetics and 1 for diabetics. Diabetes is a significant risk factor for heart disease.
- BMI (Body Mass Index); A measure of body fat based on height and weight, calculated as weight in kilograms divided by height in meters squared. A high BMI can indicate obesity, which is a risk factor for heart disease.
- High Blood Pressure; Indicates whether the individual has been diagnosed with high blood pressure, typically coded as 0 for no and 1 for yes. This is a direct risk factor for heart disease.
- Low LDL Cholesterol; Refers to low levels of high-density lipoprotein (HDL) cholesterol, often considered "good" cholesterol. Low HDL levels can increase the risk of heart disease.
- High LDL Cholesterol; Refers to high levels of low-density lipoprotein (LDL) cholesterol, often considered "bad" cholesterol. High LDL levels are associated with an increased risk of heart disease.
- Alcohol Consumption; Indicates the amount of alcohol consumed by the individual, often measured in standard drinks per week. Excessive alcohol consumption can contribute to heart disease.
- Stress Level; Represents the individual's perceived level of stress, which can be measured through various scales. High stress levels can negatively impact heart health.
- Sleep Hour; Indicates the average number of hours the individual sleeps per night. Poor sleep quality and insufficient sleep are linked to an increased risk of heart disease.
- Sugar Consumption; Refers to the amount of sugar consumed by the individual, often measured in grams per day. High sugar intake can contribute to obesity and diabetes, increasing heart disease risk.
- Triglyceride Level; Represents the level of triglycerides in the blood, measured in mg/dL. High triglyceride levels can increase the risk of heart disease.
- Fasting Blood Sugar; Indicates the blood sugar level after fasting, measured in mg/dL. Elevated fasting blood sugar levels can indicate diabetes or prediabetes, which are risk factors for heart disease.
- CRP Level (C-Reactive Protein Level); Measures the level of C-reactive protein in the blood, which can indicate inflammation. High CRP levels are associated with an increased risk of heart disease.
- Homocystein Level; Refers to the level of homocysteine in the blood, an amino acid that, at elevated levels, can increase the risk of heart disease.
- Heart Disease Status; Indicates whether the individual has been diagnosed with heart disease, typically coded as 0 for no heart disease and 1 for heart disease. This is the target variable for predictive modeling.

```
In [7]: # Let's check for missing values in our data  
data.isna().sum()
```

```
Out[7]: Age                29  
Gender                19  
Blood Pressure        19  
Cholesterol Level      30  
Exercise Habits        25  
Smoking               25  
Family Heart Disease   21  
Diabetes              30  
BMI                  22  
High Blood Pressure    26  
Low HDL Cholesterol    25  
High LDL Cholesterol   26  
Alcohol Consumption    32  
Stress Level          22  
Sleep Hours           25  
Sugar Consumption      30  
Triglyceride Level     26  
Fasting Blood Sugar    22  
CRP Level             26  
Homocysteine Level     20  
Heart Disease Status   0  
dtype: int64
```

All columns in our data have missing values except the `Heart Disease Status` column.

```
In [8]: # Let's check for duplicates in our data  
data.duplicated().sum()
```

```
Out[8]: 0
```

Our data has no duplicates

DATA CLEANING

- Under data cleaning we'll handle missing values in our data and also check for outliers.

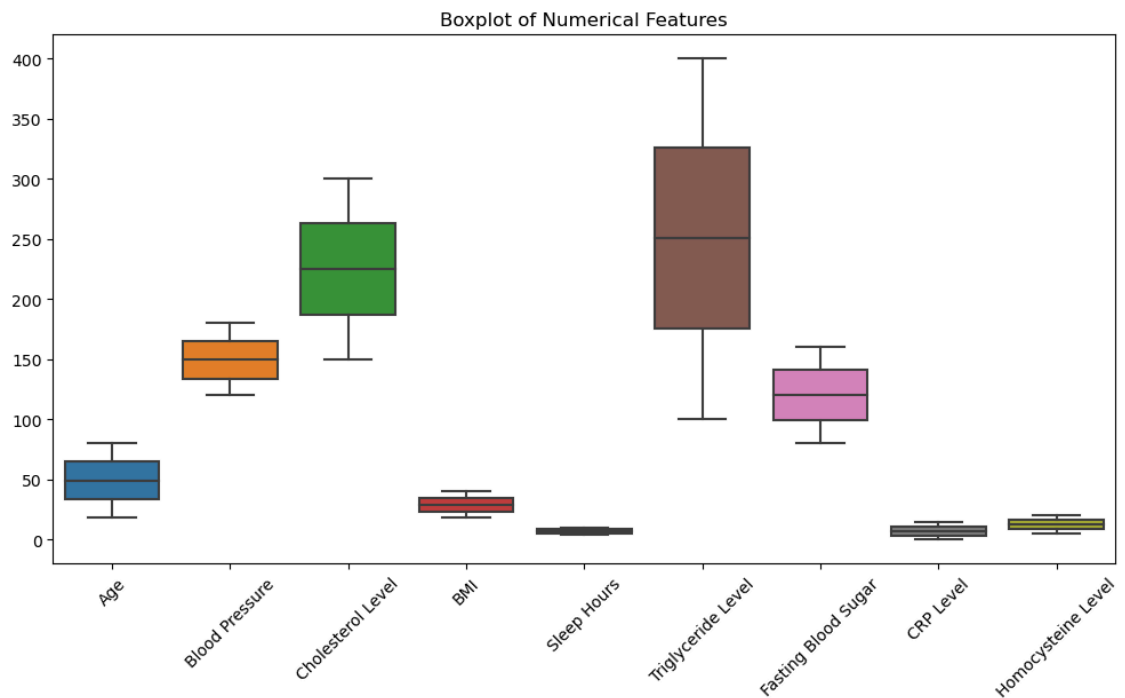
```
In [9]: # Data Cleaning  
# Handling missing values  
imputer = SimpleImputer(strategy='mean')  
numerical_cols = data.select_dtypes(include=['float64', 'int64']).columns  
data[numerical_cols] = imputer.fit_transform(data[numerical_cols])  
  
data.dropna(inplace=True) # Drop remaining categorical missing values
```

```
In [10]: # Let's check if we've handled the missing values  
data.isna().sum()
```

```
Out[10]: Age                                0  
Gender                                0  
Blood Pressure                        0  
Cholesterol Level                    0  
Exercise Habits                      0  
Smoking                             0  
Family Heart Disease                 0  
Diabetes                            0  
BMI                                  0  
High Blood Pressure                  0  
Low HDL Cholesterol                  0  
High LDL Cholesterol                 0  
Alcohol Consumption                  0  
Stress Level                         0  
Sleep Hours                         0  
Sugar Consumption                    0  
Triglyceride Level                   0  
Fasting Blood Sugar                  0  
CRP Level                           0  
Homocysteine Level                   0  
Heart Disease Status                 0  
dtype: int64
```

Now we have a clean data with no missing values.


```
In [11]: ▶ # Boxplot for numerical features
plt.figure(figsize=(12,6))
sns.boxplot(data=data.select_dtypes(include=['float64', 'int64']))
plt.title('Boxplot of Numerical Features')
plt.xticks(rotation=45)
plt.show()
```

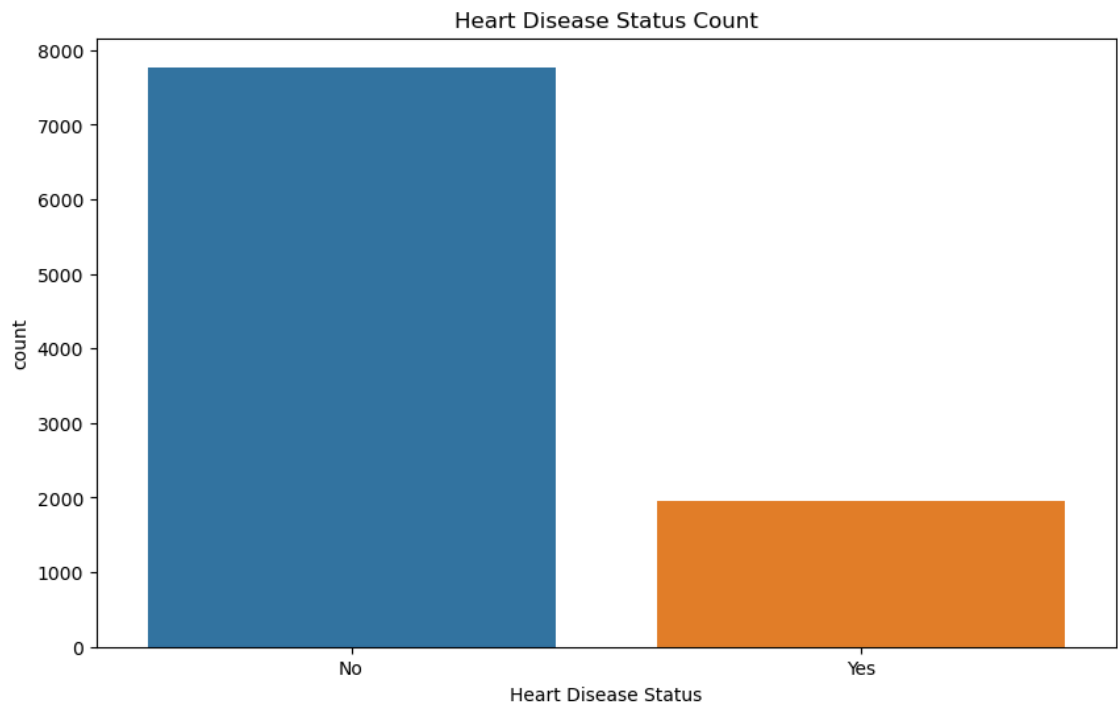


By the use of boxplot we are able to see that our numerical columns have no outliers.

EXPLORATORY DATA ANALYSIS (EDA)

- Visualizing the Distribution of Target Variable

```
In [12]: ▶ plt.figure(figsize=(10,6))  
sns.countplot(x='Heart Disease Status', data=data)  
plt.title('Heart Disease Status Count')  
plt.show()
```



A count plot visualizes the distribution of patients with and without heart disease. This helps determine whether the dataset is balanced or imbalanced. From the visualization we can see that our data is imbalanced thus it will need to be balanced.

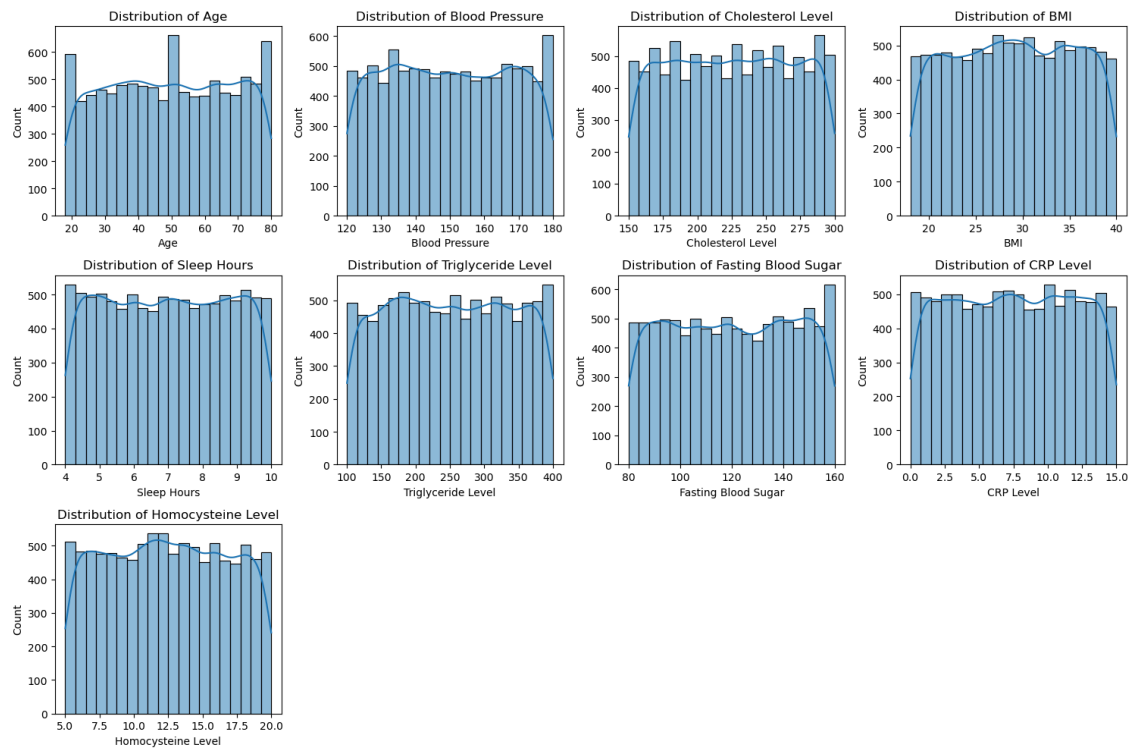
- Visualizing the Distribution of Numerical Features

```
In [13]: # Plotting histograms for numerical features
numerical_features = data.select_dtypes(include=[np.number]).columns.tolist()

# Setting up the matplotlib figures
plt.figure(figsize=(15, 10))

for i, feature in enumerate(numerical_features):
    plt.subplot(3, 4, i + 1)
    sns.histplot(data[feature], bins=20, kde=True)
    plt.title(f'Distribution of {feature}')

plt.tight_layout()
plt.show()
```



Histograms are plotted for all numerical features. These help identify skewness, normality, and the presence of outliers. By the use of boxplot we saw that our data has no outliers and by the use of histograms we can see that our numerical features are normally distributed.

- Analyze Categorical Features in comparison with the target variable

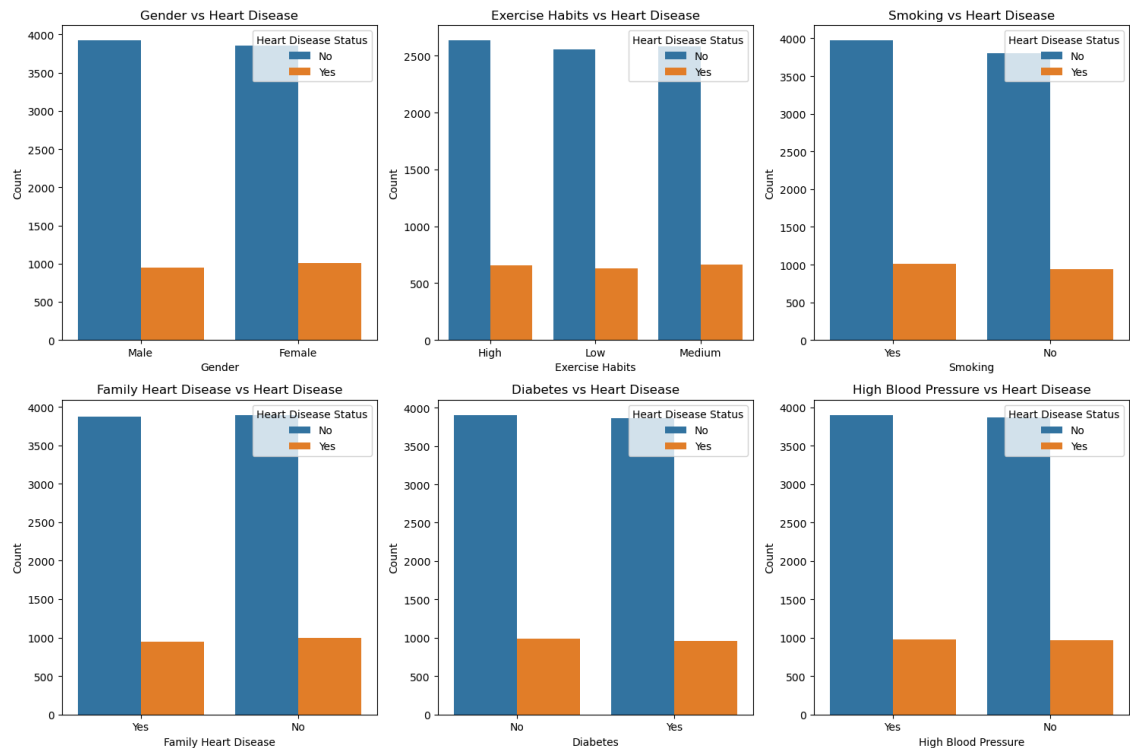
```
In [14]: # Visualizing the relationship between categorical features and the target
categorical_features = data.select_dtypes(include=['object']).columns.tolist()

# Limiting to the first 6 features for visualization
categorical_features = categorical_features[:6]

# Setting up the matplotlib figure
plt.figure(figsize=(15, 10))

for i, feature in enumerate(categorical_features):
    plt.subplot(2, 3, i + 1)
    sns.countplot(x=feature, hue='Heart Disease Status', data=data)
    plt.title(f'{feature} vs Heart Disease')
    plt.xlabel(feature)
    plt.ylabel('Count')

plt.tight_layout()
plt.show()
```



Categorical variables are analyzed using count plots. This helps determine if certain categories are more associated with heart disease. From the visualizations we can observe that patients who smoke are higher than those who don't smoke in regards to heart disease. The same applies to those with high blood pressure.

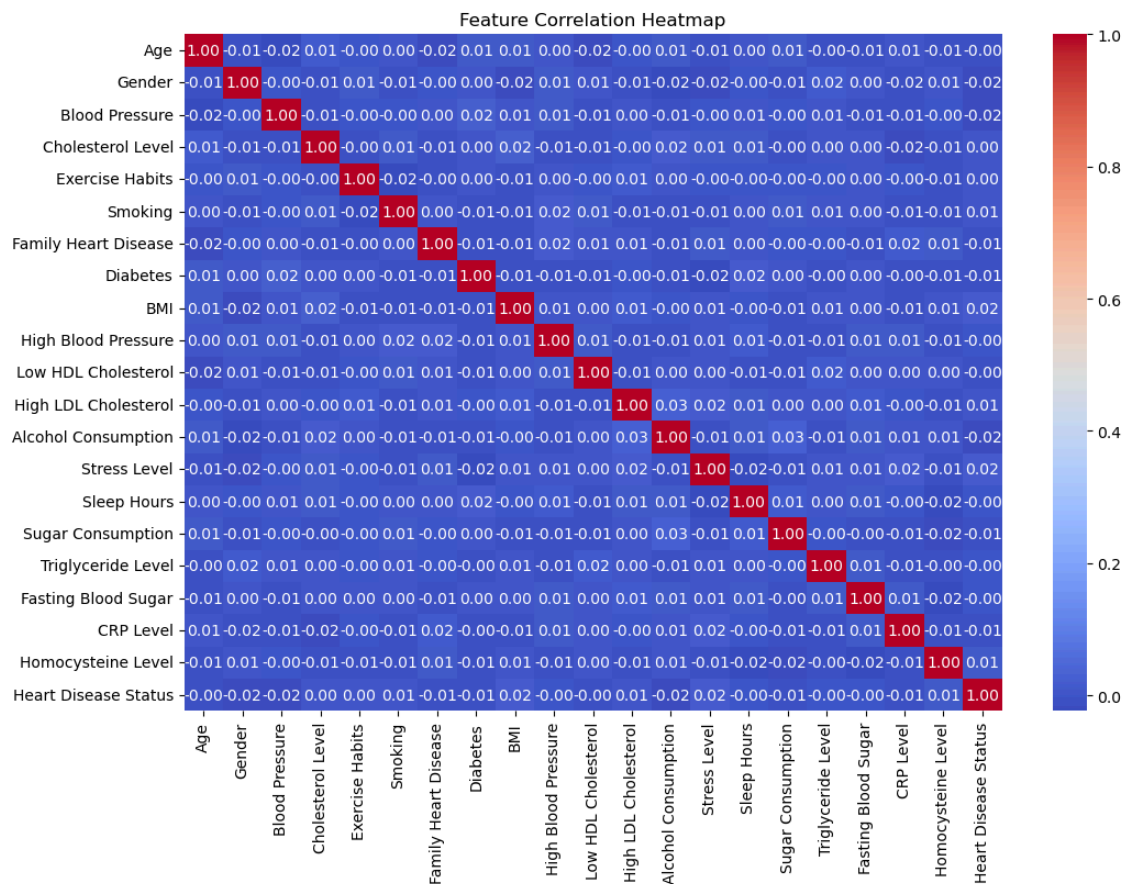
- Encoding categorical variables

```
In [15]: ▶ label_encoders = {}
categorical_cols = data.select_dtypes(include=['object']).columns
for col in categorical_cols:
    le = LabelEncoder()
    data[col] = le.fit_transform(data[col])
    label_encoders[col] = le
```

Label Encoding is a technique used to convert categorical variables into numerical values. Works well with ordinal categorical data (e.g., "Low", "Medium", "High" for our exercise column).

- Correlation heatmap

```
In [16]: ▶ plt.figure(figsize=(12,8))
sns.heatmap(data.corr(), annot=True, cmap='coolwarm', fmt='.2f')
plt.title('Feature Correlation Heatmap')
plt.show()
```



A heatmap is used to examine features with a strong correlation to heart disease e.g., cholesterol, exercise habits, smoking, e.t.c. These features will help us know the main risk factors leading to heart disease.

```
In [17]: ▶ # Display feature correlation with target variable in ascending order
correlation_with_target = data.corr()['Heart Disease Status'].drop('Heart Disease Status')
print("\nFeature Correlation with Target (Ascending Order):\n", correlation_with_target)
```

Feature Correlation with Target (Ascending Order):

Gender	-0.016351
Blood Pressure	-0.016319
Alcohol Consumption	-0.016121
Sugar Consumption	-0.011651
Family Heart Disease	-0.008118
CRP Level	-0.005728
Diabetes	-0.005032
Low HDL Cholesterol	-0.004940
Age	-0.004230
Fasting Blood Sugar	-0.001704
Sleep Hours	-0.001089
Triglyceride Level	-0.000453
High Blood Pressure	-0.000209
Cholesterol Level	0.001185
Exercise Habits	0.004771
Smoking	0.005945
High LDL Cholesterol	0.008030
Homocysteine Level	0.008516
Stress Level	0.017294
BMI	0.019544

Name: Heart Disease Status, dtype: float64

DATA PREPARATION

In this section we will prepare and build models that will predict our outcomes. Some of the steps to be taken will involve:

1. Defining the features and target variable for model training
2. Handling class imbalances for our models.
3. Splitting the data into training and testing set.
4. Training multiple models on the training dataset.
5. Identifying the best performing models using different metrics
6. Tuning the best model using hyperparameter tuning.

This will enable us to ascertain whether its likely to predict the likelihood of individuals knowing if they are at risk of being diagnosed with Heart Disease or not

```
In [18]: ▶ # Data Preparation
X = data.drop(columns=['Heart Disease Status']) # Features
y = data['Heart Disease Status'] # Target
```

```

In [19]: ▶ # Handling Imbalance using SMOTE
          smote = SMOTE(random_state=42)
          X_resampled, y_resampled = smote.fit_resample(X, y)

In [20]: ▶ # Train-test split
          X_train, X_test, y_train, y_test = train_test_split(X_resampled, y_resampled,

In [21]: ▶ # Feature scaling
          scaler = StandardScaler()
          X_train = scaler.fit_transform(X_train)
          X_test = scaler.transform(X_test)

```

MODELING

- **Random Forest**

```

In [22]: ▶ rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
          rf_model.fit(X_train, y_train)

```

Out[22]: RandomForestClassifier(random_state=42)

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.

On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

```

In [23]: ▶ y_pred_rf = rf_model.predict(X_test)
          print('\nRandom Forest Model')
          print('Accuracy:', accuracy_score(y_test, y_pred_rf))
          print(classification_report(y_test, y_pred_rf))

```

Random Forest Model

Accuracy: 0.8025088452878739

	precision	recall	f1-score	support
0	0.77	0.86	0.81	1551
1	0.84	0.75	0.79	1558
accuracy			0.80	3109
macro avg	0.81	0.80	0.80	3109
weighted avg	0.81	0.80	0.80	3109

The Random Forest model achieved the following performance metrics:

- Accuracy: 0.80 (80.26%)

This indicates that 80% of the total predictions were correct.

- Precision:

Class 0: 0.77

Class 1: 0.84

This measures the proportion of positive predictions that were actually correct. Class 1 has a higher precision, meaning fewer false positives.

- Recall:

Class 0: 0.86

Class 1: 0.75

Recall measures the proportion of actual positives correctly identified. Class 0 has a higher recall, meaning fewer false negatives.

- F1-Score:

Class 0: 0.81

Class 1: 0.79

The F1-score balances precision and recall. The model performs fairly well, with both classes having an F1-score close to 0.80.

- ***Logistic Regression***

```
In [24]: ▶ lr_model = LogisticRegression(max_iter=500)
          lr_model.fit(X_train, y_train)
```

Out[24]: LogisticRegression(max_iter=500)

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.

On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.


```
In [25]: ► y_pred_lr = lr_model.predict(X_test)
print('\nLogistic Regression Model')
print('Accuracy:', accuracy_score(y_test, y_pred_lr))
print(classification_report(y_test, y_pred_lr))
```

```
Logistic Regression Model
Accuracy: 0.7143776133805082
```

	precision	recall	f1-score	support
0	0.71	0.72	0.72	1551
1	0.72	0.71	0.71	1558
accuracy			0.71	3109
macro avg	0.71	0.71	0.71	3109
weighted avg	0.71	0.71	0.71	3109

The Logistic Regression Model achieved the following performance metrics:

- Accuracy: 0.71 (71.44%)

This means that 71.44% of the total predictions were correct.

- Precision (measures the proportion of predicted positives that are actually positive):

Class 0: 0.71

Class 1: 0.72

The model has nearly balanced precision for both classes, meaning it has a similar rate of false positives for both.

- Recall (measures the proportion of actual positives correctly identified):

Class 0: 0.72

Class 1: 0.71

The recall values are also very close, meaning the model captures a similar proportion of actual positives in both classes.

- F1-Score (harmonic mean of precision and recall):

Class 0: 0.72

Class 1: 0.71

The F1-scores indicate balanced performance between precision and recall for both classes.

- **GradientBoostingClassifier Model**

```
In [26]: ► gb_model = GradientBoostingClassifier(n_estimators=100, random_state=42)
gb_model.fit(X_train, y_train)
```

Out[26]: GradientBoostingClassifier(random_state=42)

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.

On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

```
In [27]: ► y_pred_gb = gb_model.predict(X_test)
print('\nGradientBoostingClassifier Model')
print('Accuracy:', accuracy_score(y_test, y_pred_gb))
print(classification_report(y_test, y_pred_gb))
```

GradientBoostingClassifier Model

Accuracy: 0.7561917015117401

	precision	recall	f1-score	support
0	0.75	0.77	0.76	1551
1	0.76	0.74	0.75	1558
accuracy			0.76	3109
macro avg	0.76	0.76	0.76	3109
weighted avg	0.76	0.76	0.76	3109

The Gradient Boosting Classifier Model achieved the following performance metrics:

- Accuracy: 0.756 (75.62%)

This means the model correctly predicted 75.62% of the test data.

- Precision (measures the proportion of predicted positives that are actually positive):

Class 0: 0.75

Class 1: 0.76

The precision values are balanced, indicating a similar false positive rate for both classes.

- Recall (measures the proportion of actual positives correctly identified):

Class 0: 0.77

Class 1: 0.74

Class 0 has a slightly higher recall, meaning it correctly identifies more true positives compared to Class 1.

- F1-Score (harmonic mean of precision and recall):

Class 0: 0.76

Class 1: 0.75

The F1-scores indicate that the model performs consistently across both classes

- **Support Vector Classifier Model**

```
In [28]:  svc_model = SVC()
          svc_model.fit(X_train, y_train)
```

Out[28]: SVC()

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.

On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

```
In [29]:  y_pred_svm = svc_model.predict(X_test)
          print('\nsvc Model')
          print('Accuracy:', accuracy_score(y_test, y_pred_svm))
          print(classification_report(y_test, y_pred_svm))
```

```
svc Model
Accuracy: 0.7680926342875523
```

	precision	recall	f1-score	support
0	0.74	0.82	0.78	1551
1	0.80	0.72	0.76	1558
accuracy			0.77	3109
macro avg	0.77	0.77	0.77	3109
weighted avg	0.77	0.77	0.77	3109

The Support Vector Machine (SVM) Model achieved the following performance metrics:

- Accuracy: 0.768 (76.81%)

This means the model correctly predicted 76.81% of the test data.

- Precision (proportion of predicted positives that are actually positive):

Class 0: 0.74

Class 1: 0.80

The model is more precise in predicting Class 1 (positive cases) than Class 0.

- Recall (proportion of actual positives correctly identified):

Class 0: 0.82

Class 1: 0.72

Class 0 has a higher recall, meaning it correctly identifies more true negatives, while Class 1 has a slightly lower recall.

- F1-Score (harmonic mean of precision and recall):

Class 0: 0.78

Class 1: 0.76

The F1-scores indicate that Class 0 has a slightly better balance between precision and recall compared to Class 1.

Best model selection

The Random Forest model achieved an accuracy of 80.26%, which has the best accuracy compared to the other models.

The classification report shows that the Random Forest model provides a good balance between precision and recall.

We shall therefore go ahead and tune the Random Forest Model to improve its performance

ADVANCED MACHINE LEARNING (Hyperparameter Tuning)

```
In [30]: param_grid = {  
    'n_estimators': [50, 100, 200],  
    'max_depth': [10, 20, None],  
    'min_samples_split': [2, 5, 10]  
}  
  
grid_search = GridSearchCV(RandomForestClassifier(random_state=42), param_  
grid_search.fit(X_train, y_train)
```

```
Out[30]: GridSearchCV(cv=5, estimator=RandomForestClassifier(random_state=42),  
    param_grid={'max_depth': [10, 20, None],  
    'min_samples_split': [2, 5, 10],  
    'n_estimators': [50, 100, 200]},  
    scoring='accuracy')
```

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.

On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

```
In [31]: # Best Model Evaluation
best_rf_model = grid_search.best_estimator_
y_pred_best = best_rf_model.predict(X_test)

best_accuracy = accuracy_score(y_test, y_pred_best)
best_auc = roc_auc_score(y_test, y_pred_best)
best_report = classification_report(y_test, y_pred_best)

print(f'\nBest Random Forest Model Accuracy: {best_accuracy}')
print(f'Best ROC AUC Score: {best_auc}')
print('Best Model Classification Report:\n', best_report)
```

```
Best Random Forest Model Accuracy: 0.8076551945963333
Best ROC AUC Score: 0.8077831272051904
Best Model Classification Report:
              precision    recall  f1-score   support

     0       0.78        0.86        0.82        1551
     1       0.85        0.75        0.80        1558

 accuracy                   0.81        3109
 macro avg       0.81        0.81        0.81        3109
 weighted avg    0.81        0.81        0.81        3109
```

The Random Forest model after hyperparameter tuning achieved the following metrics:

- Accuracy: 0.807 (80.77%)

This means the model correctly classified 80.77% of test cases, making it the best-performing model so far in terms of accuracy.

- ROC AUC Score: 0.807 (80.73%)

This indicates good discrimination between positive and negative classes.

- Precision (Proportion of predicted positives that are actually positive):

Class 0: 0.78

Class 1: 0.85

The model is more precise in predicting Class 1 (positive cases) than Class 0.

- Recall (Proportion of actual positives correctly identified):

Class 0: 0.86 (High recall, meaning fewer false negatives)

Class 1: 0.75

- F1-Score (Harmonic mean of precision and recall):

Class 0: 0.82

Class 1: 0.80

The model balances precision and recall well for both classes.

CONCLUSION

This project aimed to analyze and predict heart disease based on various health-related features. Through Exploratory Data Analysis (EDA), feature engineering, and model evaluation, we derived the following key insights:

1. Dataset Imbalance

- The count plot of patients with and without heart disease revealed an imbalanced dataset. Since an imbalanced dataset can negatively affect model performance, SMOTE (Synthetic Minority Over-sampling Technique) was applied to balance the classes before training machine learning models.

2. Data Distribution and Outliers

- Histogram analysis showed that the numerical features are normally distributed, which is beneficial for machine learning models that assume normality.
- Boxplots confirmed that there were no significant outliers, ensuring that extreme values would not distort predictions.

3. Key Risk Factors for Heart Disease

- A heatmap and feature correlation analysis revealed that features such as cholesterol levels, exercise habits, smoking, and stress levels had the strongest correlation with heart disease. These factors play a critical role in determining a patient's heart disease risk.

4. Feature Encoding

- Label Encoding was applied to transform categorical variables into numerical form. This was essential for incorporating ordinal categorical variables, such as exercise habits, into machine learning models.

5. Feature Importance and Correlation

- The correlation analysis showed BMI, Stress Level, and Homocysteine Level as the highest positively correlated features with heart disease, while Gender, Blood Pressure, and Alcohol Consumption had the lowest correlation.

6. Model Performance Comparison

- Four machine learning models—Random Forest, Logistic Regression, Gradient Boosting, and SVM—were trained and evaluated.
- Random Forest emerged as the best model with an accuracy of 80.25%.
- After hyperparameter tuning and advanced machine learning techniques, the Random Forest model's accuracy improved from 80.25% to 80.77% with an ROC score of 0.8, indicating strong predictive capability.

RECOMMENDATIONS

Based on these findings, the following recommendations are suggested:

1. Public Health Interventions

- Smoking, cholestral level, exercise habits, BMI, stress level and Homocysteine Level are strong indicators of heart disease risk. Awareness programs and lifestyle modification campaigns should be implemented to reduce these risk factors in at-risk populations.

2. Government interventions

- The government should come up with strict punishment such as fines for people who smoke in undesignated places. They can also advice manufacturers to consider filters which are less harmful to the smokers.
- The government should allocate more funds in the health sector to facilitate frequent health camps and mobile clinics which can help in detecting heart diseases early on.
- Additionally they should advice processors to correctly label ingredients such as quantity of sugar for consumer goods, this will aid consumers in their decision-making.

3. Clinical Application of the Model

- The improved Random Forest model (80.77% accuracy, 0.8 ROC score) can be integrated into a clinical decision support system (CDSS) to assist healthcare providers in identifying high-risk patients and making data-driven decisions.

4. Feature Refinement and Further Data Collection

- The correlation analysis showed that some features had minimal impact on heart disease prediction. Future research should focus on acquiring more relevant patient data, such as genetic factors or real-time heart monitoring metrics, to enhance model accuracy.

5. Deploying the Model in a Real-World Setting

- The model can be deployed as a web-based or mobile health application where users can input their health metrics and receive a risk assessment for heart disease. This could serve as an early warning system, prompting individuals to seek medical attention when necessary.

6. Continued Model Optimization

- Further improvements can be achieved through deep learning models, feature selection techniques, and real-time data analysis. Additionally, testing the model on external datasets will help validate its generalizability across different populations.

References

- kontchou, N.-A. T., mcCracy, A. W., & Schulman, K. A. (2019). Workforce Cost Model for Expanding Congenital and Rheumatic Heart Disease Services in Kenya. World J Pediatr Congenit Heart Surg, 321-327 .

- Mbau, L., Jean M Fourie, W. s., Scarlatescu, O., Nel, G., & Gathecha, G. (2021, May/June). PASCAR and WHF Cardiovascular Disease Scorecard Project. Cardiovascular Journal of Heart Africa, 32(3). Retrieved from <https://world-heart-federation.org/wp-content/uploads/Kenya-Country-Report.pdf> (<https://world-heart-federation.org/wp-content/uploads/Kenya-Country-Report.pdf>)